

Practical 7

Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

```
GNU nano 8.7 banker.c
#include <stdio.h>

#define P 5    // Number of Processes
#define R 4    // Number of Resources

int main()
{
    int allocation[P][R], max[P][R], need[P][R];
    int available[R];
    int finish[P] = {0};
    int safeSeq[P];
    int work[R];
    int i, j, k, count = 0;

    printf("\nEnter Allocation Matrix (5x4):\n");
    for(i = 0; i < P; i++)
    {
        for(j = 0; j < R; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Maximum Matrix (5x4):\n");
    for(i = 0; i < P; i++)
    {
        for(j = 0; j < R; j++)
        {
            scanf("%d", &max[i][j]);
        }
    }

    // Banker's Algorithm
    while(count < P)
    {
        int found = 0;
        for(i = 0; i < P; i++)
        {
            if(finish[i] == 0)
            {
                for(j = 0; j < R; j++)
                {
                    if(need[i][j] > work[j])
                        break;
                }

                if(j == R)
                {
                    for(k = 0; k < R; k++)
                        work[k] += allocation[i][k];

                    safeSeq[count] = i;
                    count++;
                    finish[i] = 1;
                }
            }
        }
    }

    printf("\nSafe Sequence: ");
    for(i = 0; i < count; i++)
        printf("%d ", safeSeq[i]);
    printf("\n");
}
```

^G Help ^O Write Out ^F where Is ^K Cut
^X Exit ^R Read File ^\ Replace ^U Paste

```

        printf("\nSystem is NOT in Safe State (Deadlock)");
        return 0;
    }

    printf("\nSystem is in SAFE State\nSafe Sequence:\n");

    char *processes[] = {
        "Google Drive",
        "Firefox",
        "Word Processor",
        "Excel",
        "PowerPoint"
    };

    for(i = 0; i < P; i++)
    {
        printf("%s", processes[safeSeq[i]]);
        if(i != P-1)
            printf(" -> ");
    }

    printf("\n");

    return 0;
}

```

```

PRACHI@LAPTOP-CVJL7P0G UCRT64 ~
$ nano banker.c

PRACHI@LAPTOP-CVJL7P0G UCRT64 ~
$ gcc banker.c -lpthread -o banker
./banker
Enter Allocation Matrix (5x4):
1 0 1 2
0 1 0 1
1 1 0 0
0 0 1 1
1 0 0 2
Enter Maximum Matrix (5x4):
2 1 1 3
1 2 1 2
1 1 1 1
0 1 2 2
2 1 1 3
Enter Available Resources (4):
1 1 2 1

Need Matrix:
1 1 0 1
1 1 1 1
0 0 1 1
0 1 1 1
1 1 1 1

System is in SAFE State.
Safe Sequence:
Google Drive -> Firefox -> Word Processor -> Excel -> PowerPoint

PRACHI@LAPTOP-CVJL7P0G UCRT64 ~
$ |

```